

Infrastructure as Code

AWS CloudFormation

Ron C. Chiang

University of St. Thomas



Agenda

- Infrastructure as Code (IaC)
- Dynamic infrastructure platforms
- Infrastructure definition tools
- JSON/ YAML
- AWS CloudFormation

Infrastructure as Code

- Definition: **An approach to infrastructure automation based on practices from software development.**
 - Version control systems
 - Automated code testing
 - Continuous integration

Infrastructure as Code Principles

- Systems can be easily reproduced like software builds.
 - A build creates artifacts that can be thrown away and recreated.
- Systems are disposable.
 - Treat your servers like cattle, not pets.
- Systems are consistent.
 - Same OS/distribution, packages, configuration.
- Recognize that infrastructure will always need to change.
- We use infrastructure definition and server configuration management tools to support these principles.
 - AWS Cloudstack, Terraform
 - Chef, Puppet, Ansible, Salt

Cattle vs. Pets

- How do we know if we are treating our servers like pets?
 - Servers are named after things – Greek gods, planets, comic book characters – instead of function.
 - The server stays running for months or years.
 - The server is fixed and nursed back to health if it fails.
 - People are afraid to replace the server.
 - Beware of pet cows!



Infrastructure as Code Practices

- Use definition files
 - Specify infrastructure elements and how they are configured
 - JSON, YAML, XML or Domain Specific Language (DSL)
 - Definition files and scripts offer a testable form of documentation.
- Version everything
 - Source of truth for the desired state of infrastructure.
 - Traceability: history of changes and who made them.
 - Rollback: ability to revert infrastructure to a previous state.
 - Visibility: infrastructure configuration is clearly visible to everyone.
- Continuously test systems
 - Automated infrastructure testing: every infrastructure change goes through a testing process.
 - Build infrastructure pipelines

Anti-Fragility

- IT has always been focused on building reliable infrastructure.
- Infrastructure as Code goes beyond traditional IT and makes infrastructure **anti-fragile**.
 - Exercise damages muscles, but also makes them stronger.
 - Preventing change in IT systems makes them weaker, while increasing and managing change makes them stronger.
- Netflix Chaos Monkey & Chaos Kong



Chaos Engineering

- Large-scale distributed systems are becoming increasingly complex.
- Complex systems fail in unpredictable ways.
- Chaos engineering seeks to identify weaknesses in complex systems.
 - Improper fallback settings when a system is unavailable.
 - Retry storms from improperly tuned timeouts.
 - Outages when downstream dependencies receive too much traffic.
 - Cascading failures when a single point of failure crashes.
- Try to validate steady-state hypothesis through experimentation.
- Site: <http://principlesofchaos.org>

“Death Star” Architecture Diagrams



Netflix



Gilt Groupe (12 of 450)



Twitter

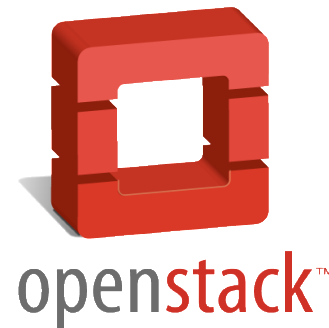
As visualized by Appdynamics, Boundary.com and Twitter internal tools

Building Infrastructure as Code

- We need three capabilities to build and maintain dynamic infrastructure:
 - **Dynamic infrastructure platform**
 - Provides an environment in which to dynamically build infrastructure components.
 - **Infrastructure definition tools**
 - Provides a way to describe how to build components in the dynamic infrastructure platform.
 - **Configuration management tools**
 - Provides a way to manage the configuration of infrastructure components over time.

Dynamic Infrastructure Platform

- A system that provides servers, storage, and networking infrastructure in a programmatic way.
 - Amazon Web Services
 - Microsoft Azure
 - Google Compute Engine
 - OpenStack
 - VMware vCloud
- Requirements:
 - Access to an API
 - On-demand: create and destroy resources immediately.
 - Shouldn't require requests to a provider to provision infrastructure.
 - **Anti-pattern: Hand-cranked clouds**



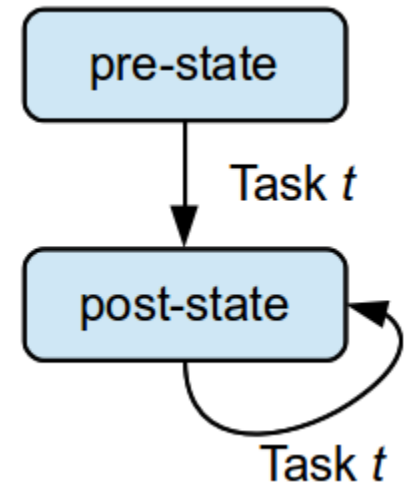
Infrastructure Definition Tools

- Tool that allows people to specify what infrastructure component to create and how to configure it.
 - Leverages the API provided by the dynamic infrastructure platform to create the component.
 - Examples:
 - AWS Cloud Formation
 - Terraform
 - Openstack Heat

Infrastructure Definition Tool

Characteristics

- **Idempotent:** execute the same script multiple times without negatively impacting infrastructure.
- **Pre/post-checks:** ability for tool to validate starting conditions and check for successful completion.
- **Visible failure:** shouldn't hide provisioning failures.
- **Parameterized:** allows the reuse of infrastructure definitions for multiple use-cases (i.e., environments).



Using Infrastructure Definition Tools

- Common practices:
 - **Unattended execution:** definition tools do not require human intervention to build infrastructure.
 - **Configuration definition files:** infrastructure definitions are stored in text files (JSON, YAML, DSL, etc.)
 - **Externalized configuration:** code that defines infrastructure is located outside the tool versus some sort of hidden internal configuration file.
 - **VCS:** infrastructure definition files are stored in version control systems.

Configuration Definition Files

- Two ways to define infrastructure in configuration files:
 - **Procedurally**: describe the step-by-step process required to build infrastructure
 - “First do X, then do Y.”
 - Example: Shell script or application library using AWS API to create infrastructure (Python Boto3).
 - **Declaratively**: describe what the infrastructure should look like and let a tool build it.
 - “It should be Z.”
 - Sometimes called policy-based configuration.
 - Benefit: typically run idempotently – can apply configuration over-and-over again.

Procedural vs. Declarative

Comparison

Imperative Shell Code

```
if [ 0 -ne $(getent passwd elmo > /dev/null)$? ]
then
    useradd elmo --gid sysadmin -n
fi

GID=$(getent passwd elmo | awk -F: '{print $4}')
GROUP=$(getent group $GID | awk -F: '{print $1}')

if [ "$GROUP" != "$GID" ] && [ "$GROUP" != "sysadmin" ]
then
    usermod --gid $GROUP $USER
fi
```

```
if [ "`getent group sysadmin | awk -F: '{print $1}'`" == "" ]
then
    groupadd sysadmin
fi
```

Declarative Puppet Code

```
user { 'elmo':
  ensure => present,
  gid    => 'sysadmin',
}
```

```
group { 'sysadmin':
  ensure => present,
}
```

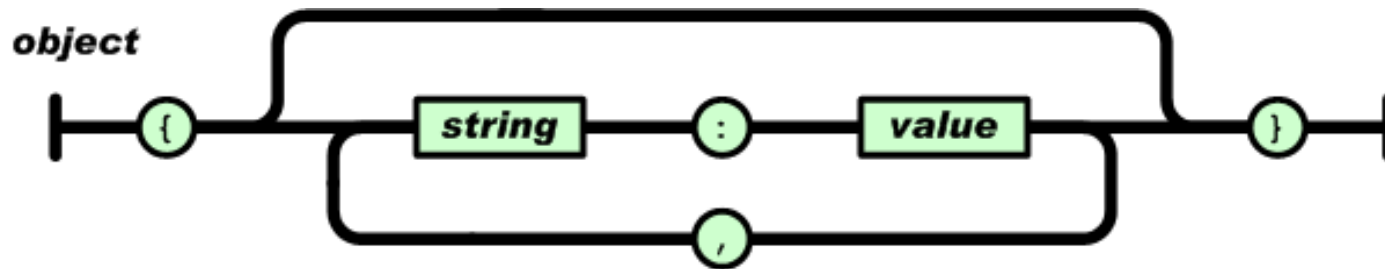
JavaScript Object Notation (JSON)

- Very light-weight data-interchange format.
- Easy for humans to read and machines to parse.
 - “self-describing”
- Based on subset of JavaScript programming language, but it’s completely language independent.
- Built on two structures:
 - Name/value pairs (object/hash)
 - Ordered lists (array)
- Values can be:
 - String, number, object, array, true, false, null

JSON Object Syntax

An **Object**:

- Unordered set of name/value pairs
- Object begins with { and ends with }
- Each name followed by : (colon) and name pairs are separated by , (comma)



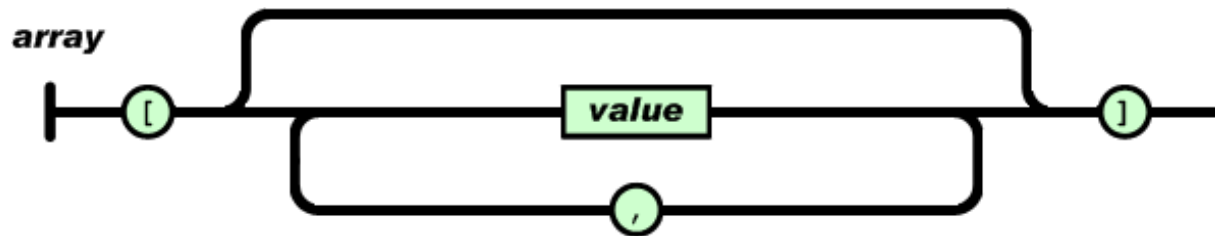
{ "name" : "Allie" }

{ "work" : "Target", "home" : "Minneapolis" }

JSON Object Syntax

An **Array**:

- Ordered collection of values
- Array begins with [and ends with]
- Values are separated by , (comma)



```
{ "employers" : [ "Target", "Cargill", "Medtronic" ] }
```

nesting objects/arrays

```
{  
  "employer" : { "name" : "United Health",  
                 "employees" : [ "John", "Miguel", "Heather" ]  
               }  
}
```

YAML

- YAML Ain't Markup Language (YAML)
- Human friendly data serialization standard for all programming languages.
- Benefits:
 - Very simple to use.
 - Human readable.
 - Information can easily be translated to multiple programming language types.

YAML Rules

- Basic rules:
 - YAML files should end with .yaml (or .yml) extension
 - YAML is case sensitive.
 - YAML does not allow tabs, use space instead.
 - Indentation (spaces) have meaning!
 - Data begins with 3 dashes “---” (optional)
- See <http://yaml.org> for specific requirements.

JSON vs. YAML Example

```
{ "breakfast_menu":  
  { "food":  
    [  
      { "name":"Belgian Waffles",  
        "price":"$6.99",  
        "description":"Our wonderful  
Belgian Waffles",  
        "calories":650  
      }  
    ]  
  }  
}
```

```
---  
breakfast_menu:  
  food:  
    - name: Belgian Waffles  
      price: "$6.99"  
      description: Our wonderful Belgian  
Waffles  
      calories: 650
```

JSONC?

- CloudFormation does not natively accept JSONC (JSON with Comments) as a template format;
- it officially supports standard JSON and YAML.
- Using JSONC, including comments (`//` or `/* */`), will result in template validation errors.
- For commenting, YAML is recommended, or comments must be placed within Metadata sections in JSON.

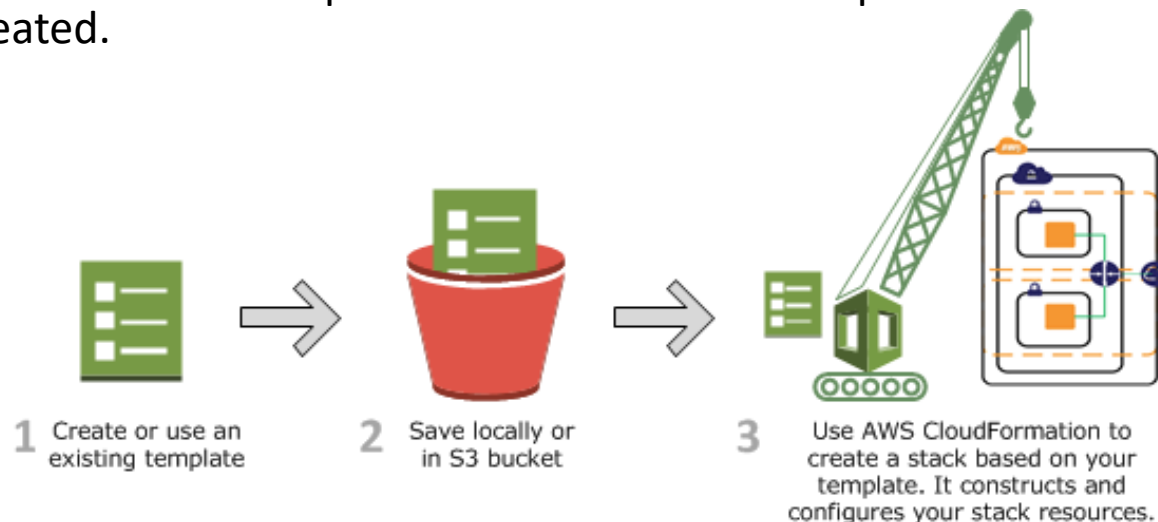
AWS CloudFormation



- AWS infrastructure definition tool used for automating the deployment of cloud infrastructure.
- Several AWS services (Elastic Beanstalk, ECS, etc.) use CloudFormation for infrastructure provisioning.
- CloudFormation uses a **template** to describe what resources to configure on AWS.
 - Template is written in JSON or YAML format.
 - Supports good Infrastructure as Code practices.
 - Store in version control
 - Ability to test
 - Repeatable
 - A set of configured cloud resources is called a **stack**.

How CloudFormation Works

- CloudFormation uses template definition to make API calls to AWS.
 - User's permissions determines what resources it can create.
- Process:
 - 1. Create a CloudFormation Template (JSON) in CloudFormation Designer or a text editor.
 - 2. Save the template locally or in an S3 bucket.
 - 3. Create a stack by specifying the template and providing any required input parameters using AWS console, API, or CLI.
 - 4. CloudFormation provisions resources and reports back when the stack is created.



CloudFormation Template

```
{  
  "AWSTemplateFormationVersion" : "version date",  
  
  "Description" : "JSON string",  
  
  "Metadata" : {  
    template metadata  
  },  
  
  "Parameters" : {  
    set of parameters  
  },  
  
  "Mappings" : {  
    set of mappings  
  },  
  
  "Conditions" : {  
    set of conditions  
  },  
  
  "Resources" : {  
    set of resources  
  },  
  
  "Outputs" : {  
    set of outputs  
  }  
}
```

Parameters section

- Templates allow the user to supply input **parameters** when launching a stack.
 - Parameter logical names must be unique.
 - Logical name definition followed by a series of parameter properties.
 - Types: String, Number, List, or AWS-specific type
 - Maximum of 60 parameters allowed in a template
- Input parameters are referenced within a template using the “**Ref**” function.

```
"Parameters" : {  
    "ParameterLogicalID" : {  
        "Type" : "DataType",  
        "ParameterProperty" : "value"  
    }  
}
```

Parameters section

- Useful parameter properties (not a complete list):
 - **AllowedValues**: an array containing list of values allowed for the parameter.
 - **Default**: the value used for the property if no value is specified.
 - **Description**: a string describing the parameter.
 - **MaxLength/MinLength/MaxValue/MinValue**: set a constraint on the input value
 - **NoEcho**: hide the parameter value in any output logs (useful for passwords).

Parameters section

```
"Parameters": {  
  "InstanceTypeParameter" : {  
    "Type" : "String",  
    "Default" : "t2.micro",  
    "AllowedValues" : ["t2.micro", "t2.small", "t2.medium"],  
    "Description" : "Enter the instance type (t2.micro, t2.small, or t2.medium)."  
  }  
},  
  
"Resources" : {  
  "Ec2Instance" : {  
    "Type" : "AWS::EC2::Instance",  
    "Properties" : {  
      "InstanceType" : { "Ref" : "InstanceTypeParameter" },  
      "ImageId" : "ami-398de234"  
    }  
  }  
}
```

Outputs section

- The optional **outputs** section declares values that CloudFormation will return when the stack is created.
 - Example: return the name of an S3 bucket after CloudFormation creates it.
 - Maximum of 60 outputs allowed in a template.
 - Available via the *describe stacks* API call.
- Value of an output can be a literal, parameter reference, pseudo parameter, mapping value, or intrinsic function.
 - It's possible to set a condition on the output using the "Condition" parameter.

```
"Outputs" : {  
  "Logical ID" : {  
    "Description" : "Information about the value",  
    "Value" : "Value to return"  
  }  
}
```

Outputs section

```
"Outputs" : {  
  "BackupLoadBalancerDNSName" : {  
    "Description": "The DNSName of the backup load balancer",  
    "Value" : { "Fn::GetAtt" : [ "BackupLoadBalancer", "DNSName" ] },  
    "Condition" : "CreateProdResources"  
  },  
  "InstanceID" : {  
    "Description": "The Instance ID",  
    "Value" : { "Ref" : "EC2Instance" }  
  }  
}
```

Intrinsic Functions

- CloudFormation supports several built-in functions to help you build stacks.
 - Conditional (if-then-else)
 - Lookup
 - Retrieve data
- Only usable in certain parts of the template – mainly in resource properties.

```
"Properties" : {  
  "MyLBDNSName" : {  
    "Fn::GetAtt" : [ "MyLoadBalancer", "DNSName" ]  
  }  
}
```

Intrinsic Functions

- **Fn::Base64**
 - Converts test string to Base64 representation.
 - Typically used for passing userdata to an EC2 instance.
 - { "Fn::Base64" : "sudo yum update -y" }
- **Fn::FindInMap**
 - Performs a value lookup in a hash/dictionary object.
 - "Fn::FindInMap" : ["MapName", "TopLevelKey", "SecondLevelKey"]
- **Fn::GetAtt**
 - Returns the value of an attribute from a resource in the template.
 - "Fn::GetAtt" : ["MyLoadBalancer", "DNSName"]
- **Fn::GetAZs**
 - Returns an array that lists Availability Zones for a specified region (or whatever region the stack is launching in).
 - { "Fn::GetAZs" : "us-east-1" }
 - { "Fn::GetAZs" : { "Ref" : "AWS::Region" } }

Intrinsic Functions

- **Fn:Join**
 - Concatenates a set of values into a single value with an optional delimiter.
 - "Fn::Join" : [":", ["a", "b", "c"]]
 - Returns: "a:b:c"
- **Fn:Select**
 - Returns a single object from a list of objects.
 - { "Fn::Select" : ["1", ["apples", "grapes", "oranges", "mangoes"]] }
 - Returns: "grapes"
- **Ref**
 - Returns the value of the specified parameter or resource.
 - "Ref" : *"logicalName"*
- **Condition Functions**
 - Used within conditions sections to conditionally create resources.
 - "Fn::Equals" : ["value_1", "value_2"]
 - "Fn::If" : [*condition_name*, *value_if_true*, *value_if_false*]
 - { "Fn::If" : ["production", "m3.xlarge", "t2.micro"] }

Pseudo Parameters

- **Pseudo Parameters** are predefined parameters provided by CloudFormation.
- List of pseudo parameters:
 - **AWS::AccountId**
 - returns AWS account ID in which stack is created
 - **AWS::NotificationARNs**
 - returns list of AWS Resource Names for stack
 - **AWS::NoValue**
 - acts like a NOOP and removes the property identifier
 - **AWS::Region**
 - returns a string representing the AWS region for the stack
 - **AWS::StackId**
 - returns ID of the stack
 - **AWS::StackName**
 - returns the name of the stack

Mappings section

- Template **mappings** allow the user to define keys that map to corresponding sets of named values.
- Commonly used for setting values based on different conditions, such as regions.
 - Example: Set a value to an AMI ID based on the region the stack is launching into.
- The “**FindInMap**” function is used in the template to find a matching value based on a key.

```
"Mappings" : {  
  "Mapping01" : {  
    "Key01" : {  
      "Name" : "Value01"  
    },  
    "Key02" : {  
      "Name" : "Value02"  
    },  
  },  
}
```

Mappings section

```
"Mappings" : {  
  "RegionMap" : {  
    "us-east-1" : { "32" : "ami-6411e20d", "64" : "ami-7a11e213" },  
    "us-west-1" : { "32" : "ami-c9c7978c", "64" : "ami-cfc7978a" },  
    "eu-west-1" : { "32" : "ami-37c2f643", "64" : "ami-31c2f645" },  
    "ap-southeast-1" : { "32" : "ami-66f28c34", "64" : "ami-60f28c32" },  
    "ap-northeast-1" : { "32" : "ami-9c03a89d", "64" : "ami-a003a8a1" }  
  }  
},  
  
"Resources" : {  
  "myEC2Instance" : {  
    "Type" : "AWS::EC2::Instance",  
    "Properties" : {  
      "ImageId" : { "Fn::FindInMap" : [ "RegionMap", { "Ref" : "AWS::Region" }, "32"] },  
      "InstanceType" : "m1.small"  
    }  
  }  
}
```

If AWS::Region is us-east-1, ImageId is set to ami-6411e20d.

Conditions section

- Template **conditions** control the creation of resources or the definition of properties.
- Conditions are defined by the template author and applied to resources.
- A condition must evaluate to true in order for the resources to get built.

Conditions section

```
"Parameters" : {  
  "EnvType" : {  
    "Description" : "Environment type.",  
    "Default" : "dev",  
    "Type" : "String",  
    "AllowedValues" : ["prod", "dev"],  
    "ConstraintDescription" : "Must specify prod or dev."  
  }  
},  
  
"Conditions" : {  
  "CreateProdResources" : { "Fn::Equals" : [{"Ref" : "EnvType"}, "prod"] }  
},  
"Resources": {  
  "MountPoint" : {  
    "Type" : "AWS::EC2::VolumeAttachment",  
    "Condition" : "CreateProdResources",  
    "Properties" : {  
      "InstanceId" : { "Ref" : "EC2Instance" },  
      "VolumeId" : { "Ref" : "NewVolume" },  
      "Device" : "/dev/sdh"  
    }  
  }  
}
```

If EnvType input parameter = prod, create the MountPoint resource.

Resources section

- Describes the AWS **resource** to be built in the stack.
 - IAM policies, VPCs, Subnets, Security Groups, EC2 instances, Elastic Load Balancers, RDS databases, S3 buckets, etc.
 - The only required section.

```
"Resources" : {  
  "Logical ID" : {  
    "Type" : "Resource type",  
    "Properties" : {  
      Set of properties  
    }  
  }  
}
```

- **DependsOn** attribute allows you to specify that a resource must wait for CloudFormation to create another resource first.

```
"DependsOn" : [ String, ... ]
```

Resources section

- Multiple ways to define properties:

```
"Properties" : {  
  "String" : "one-string-value",  
  "Number" : "123",  
  "LiteralList" : [ "first-value", "second-value" ],  
  "Boolean" : "true",  
  "ReferenceForOneValue" : { "Ref" : "MyLogicalResourceName" },  
  "FunctionResultWithFunctionParams" : {  
    "Fn::Join" : [ "%", [ "Key=", { "Ref" : "MyParameter" } ] ] }  
}
```

Resources section

```
"Resources" : {  
  "web1" : {  
    "Type" : "AWS::EC2::Instance",  
    "Properties" : {  
      "UserData" : {  
        "Fn::Base64" : {  
          "Fn::Join" : [ "", [ "Queue=", { "Ref" : "MyQueue" } ] ]  
        } },  
      "AvailabilityZone" : "us-east-1a",  
      "ImageId" : "ami-20b65349"  
    }  
  },  
  
  "MyQueue" : {  
    "Type" : "AWS::SQS::Queue",  
    "Properties" : {  
      "Tags" : [  
        {  
          "Key" : "Project",  
          "Value" : "Fido"  
        }  
      ]  
    }  
  }  
}
```

Nested Stacks

- CloudFormation stacks can be used as resources in other stacks.
- Useful for making reusable templates and segmenting resources.
- Launching a template with nested stacks will launch multiple sub-stacks.
 - Nested stack templates must be stored in an S3 bucket.
 - Deleting launching stack will delete all substacks.

Configuration state

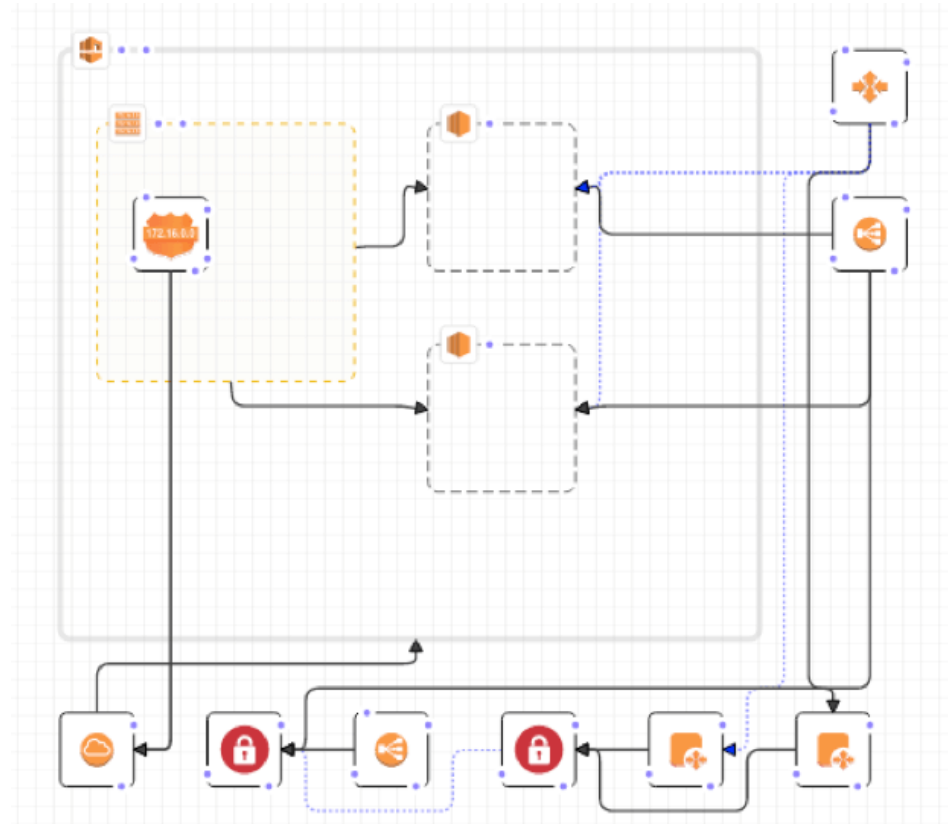
- CloudFormation does not maintain any configuration state locally.
 - It determines current state of stack anytime updates are made.
 - This allows multiple developers to work on a stack simultaneously without having to share configuration state.
- Terraform stores current configuration state locally and all developers must share information.

CloudFormation Best Practices

- Version templates in a source control system.
- Don't build a template from scratch.
 - Read sample templates or use AWS CloudFormation Designer as a starting point.
- Reuse templates as much as possible.
- Don't nest stacks more than one level.
- Use parameters to set the environment, region, instance names & sizes, etc.
 - Try not to hardcode values into resource definitions.
- Remember to add tags to all resources.
- Consider using a snapshot to backup resources that may be changed during a stack update.
 - Example: changing engine type of an RDS instance.
 - Alternatively: set a stack policy to protect resources from updates.

Infrastructure Composer

- A graphical tool to visualize and edit CloudFormation templates.
- Easy way to start new designs or edit specific resources in complex templates.
- JSON template editor supports auto-complete using Ctrl-Space.



CloudFormation Hands-on



- Launch a simple stack to create a new S3 bucket.
 - <https://seis665-public.s3.amazonaws.com/s3bucket.json>
 - Review CloudFormation template
 - Launch a new stack using the template
 - View the newly created S3 bucket.
 - Note the automatically generated bucket name.
- Modify simple S3 stack to output the bucket website address:

```
"Outputs": {  
  "BucketUrl" : {  
    "Value": {"Fn::GetAtt": ["seis665", "WebsiteURL"]},  
    "Description": "S3 bucket website address."  
  }  
}
```

CloudFormation Hands-on



- Launch a single EC2 instance into a VPC
 - VPC: 10.0.0.0/16
 - PublicSubnet: 10.0.1.0/24
 - Webserver instance
 - ImageID: ami-0440d3b780d96b29d
 - Security Group:
 - Allow port 22, 80 from anywhere
 - Review CloudFormation template
 - Launch a new stack using the template
 - View the newly created instance
- <https://seis665-public.s3.amazonaws.com/BasicWebserverInVPC.json>

AWS Cloud Development Kit

- The AWS CDK allows you to build infrastructure using popular programming languages.
 - Python, JavaScript, C#, Java
- Work with cloud resources like objects/classes and use programming constructs.
- <https://aws.amazon.com/cdk/>

Additional Resources

- AWS CloudFormation Best Practices guide
 - <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html>
- CloudFormation documentation
 - <https://aws.amazon.com/documentation/cloudformation/>
- CloudFormation Resource Types
 - <http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-template-resource-type-ref.html>